

רמזים

שים לב

שלוש דרגות בכל קושי. קרא את הדחיפה, נסה עשר דקות, ורק אז את האסטרטגיה. הכמעט-פתרון הוא הרשת, לא הקיצור.

א · "העמוד ריק ואין שגיאה"

דחיפה. פונקציה שגופה ריק מחזירה `undefined` ולא זורקת כלום. כמה מהפונקציות ב- `app.js` עדיין ריקות?

אסטרטגיה. התחל מ- `buildPads` בלבד, והתעלם מכל השאר. שים בתוכה `console.log('pads', PADS.length)` ורענן: אם רואים 7, המערך הגיע והבעיה בגוף. כשזה עובד, **מחק את השורה** — מה שעזר לך למצוא את הבעיה אינו חלק מהתיקון (שורת `code-hygiene`).

כמעט-פתרון. `buildPads` היא לולאה אחת על `PADS`, ובכל סיבוב: יוצרים `button`, יוצרים `img` ו-`span`, מצרפים אותם לכפתור עם `button.append(img, span)`, ומצרפים את הכפתור עם `padsBox.append(button)`. אין בה שום `if`.

ב · "בניתי את הכפתור ואני לא רואה אותו"

דחיפה. יצרת אלמנט. חיברת אותו למשהו?

אסטרטגיה. `document.createElement` יוצר אלמנט **מחוץ** לעמוד. הוא קיים בזיכרון וזהו, ונכנס רק כשקוראים ל- `append` עליו או על מי שמכיל אותו. בקונסולה: `el.isConnected` מחזיר `false` על אלמנט מנותק.

כמעט-פתרון. שים לב לסדר: קודם בונים את הכפתור **במלואו** ורק אז מצרפים אותו ל- `padsBox`. אם צירפת קודם והוספת אחר כך זה גם עובד, אבל אז קל לשכוח חצי.

ג · "לחיצה על פד לא עושה כלום"

דחיפה. על מה רשמת את המאזין, ומתי?

אסטרטגיה. אם רשמת בתוך הלולאה של `buildPads`, יש לך שבעה מאזינים — זה עובד, וזה לא מה שהמטלה מבקשת. אם רשמת על `#pads` **לפני** שהפדים נבנו — זה הנכון, ועובד, כי המאזין יושב על המכל ולא על הפדים.

כמעט-פתרון. מאזין אחד על `padsBox`, ובתוכו:

```
const button = event.target.closest('button.pad');
if (!button) return;
hit(button.dataset.key);
```

`event.target` הוא ה- `img` או ה- `span` כמעט תמיד, ולכן הבדיקה חייבת להיות `closest`.

ד · "המקש לא עובד / מנגן בזמן שאני מקליד שם"

דחיפה. על מה אפשר לרשום `keydown`? הפדים אינם בפקוס כשמקישים.

אסטרטגיה. `keydown` על `document` . `event.key` הוא התו — `'w'` , `'j'` אם `Shift` לחוץ, `'Enter'` . השווה אחרי `.toLowerCase()` .

כמעט-פתרון. והחצי השני, שנבדק: המאזין על `document` שומע גם הקלדה בשדה טקסט. השורה שמונעת את זה היא `if (event.target.matches('input, textarea')) return;` בתחילת המטפל. זה `event.target` בשימוש שאינו האצלה — הוא פשוט אומר לך איפה הפוקוס.

ה · "ההבהוב נשאר / לא מופיע"

דחיפה. מוסיפים מחלקה. מי מסיר אותה, ומתי?

אסטרטגיה. `classList.add('hit')` ואז `setTimeout(..., 150)` שמסיר. אם ההבהוב לא מופיע בכלל — בדוק ב-Elements אם המחלקה נוספת ומיד מוסרת, כלומר ה-`timeout` שלך הוא 0 או שקראת ל-`remove` מיד.

כמעט-פתרון. `setTimeout(() => button.classList.remove('hit'), 150);` שים לב ש-`setTimeout` מקבל **פונקציה**, לא קריאה — `button.classList.remove('hit')` בלי חץ מסביב רץ מיד, וזה בדיוק אותו באג של הסוגריים ב-`addEventListener` .

ו · "הקובייה מראה את המספר הלא נכון"

דחיפה. `rollDie()` מחזיר 1 עד 6. `DICE_FACES` מתחיל באיזה אינדקס?

אסטרטגיה. מערך מתחיל ב-0 וקובייה מתחילה ב-1. `DICE_FACES[6]` הוא `undefined` , ואז `img.src = undefined` נותן תמונה שבורה בלי שגיאה בקונסולה.

כמעט-פתרון. `dieOne.src = DICE_FACES[one - 1];` בדוק את הקצוות: גלגל עד שתקבל 1 ועד שתקבל 6, וודא ששתיהן מציגות את התמונה הנכונה. זו הסטייה באחד הקלאסית.

ז · "אני לא יודע איך לשמור סיבוב"

דחיפה. מה צריך לדעת על סיבוב כדי לצייר אותו מחדש ולהסיר אותו?

אסטרטגיה. ארבעה דברים: שני הערכים, מי ניצח, ומזהה. את המזהה אתה נותן, והוא לא חוזר על עצמו — גם לא אחרי הסרה. **אל תשתמש באינדקס כמזהה:** אחרי הסרה כל האינדקסים זזים והשורות מצביעות על הסיבוב הלא נכון.

כמעט-פתרון.

```
rounds.push({ id: nextId, one, two, winner });
nextId += 1;
```

כאשר `winner` הוא `'one'` , `'two'` או `'draw'` — לא שם השחקן. השם משתנה כשמשנים אותו בטופס, והסיבוב לא אמור להשתנות איתו.

ח · "שיניתי שם והשורות הישנות לא התעדכנו"

דחיפה. איפה השם כתוב — במערך, או בטקסט של השורה?

אסטרטגיה. אם שמרת את השם בתוך הסיבוב, כל השורות הישנות מקובעות. אם שמרת רק מי ניצח (`'one'` / `'two'`) , הטקסט בנה מחדש בכל רינדור והשמות מתעדכנים לבד.

כמעט-פתרון. פונקציה קטנה אחת שמייצרת את המשפט מסיבוב, ונקראת גם מהשורה וגם מההכרזה. כשמשנים שם — קוראים ל- `render()` ונגמר.

ט · "ההסרה עובדת רק על השורות הראשונות"

דחיפה. מתי רשמת את המאזינים, ומתי נוצרו השורות?

אסטרטגיה. מאזין נקשר לאלמנט, לא לבורר. אחרי `render` כל הכפתורים חדשים, ולכן לולאה שרצה פעם אחת מתה כולה, לא רק על השורות החדשות.

כמעט-פתרון. מאזין אחד על `#history`, שנרשם פעם אחת. `render` מרוקן את הילדים שלו ולא מחליף את ה- `<ul>` עצמו, ולכן המאזין שורד. בפנים, `closest` כמו בפדים, ואז `row.dataset.id` כדי לדעת מה לסנן.

י · "הסטטיסטיקה לא מסתדרת אחרי הסרה"

דחיפה. איך אתה יודע כמה ניצחונו יש — אתה סופר, או שאתה זוכר?

אסטרטגיה. אם יש לך `let winsOne = 0` שאתה מגדיל בכל גלגול, הסרה לא מורידה אותו, והמספר יתנפח לנצח. זה בדיוק "שני מקורות אמת" מהשיעור.

כמעט-פתרון. בכל רינדור, מהמערך: `rounds.filter((r) => r.winner === 'one').length`. שלוש שורות כאלה, ואי אפשר לסטות. ל- `#best-roll` שים לב לקצה: `Math.max()` בלי ארגומנטים הוא `-Infinity`, אז מערך ריק צריך תשובה משלו.

יא · "בקונסולה יש שגיאה אדומה על ניגון"

דחיפה. קראת ל- `playSound` מ- `data.js`, או בנית `new Audio` בעצמך?

אסטרטגיה. דפדפן רשאי לסרב לנגן, למשל לפני שהמשתמש נגע בעמוד. סירוב שלא מטופל מופיע כ- `Uncaught (in promise)` באדום, על עמוד שאין בו שום בעיה.

כמעט-פתרון. השתמש ב- `playSound` שניתן ב- `data.js`. יש בו `.catch(() => {})`. בדיוק בשביל זה, והוא נתון כדי שלא תצטרך להתעסק בזה — הבטחות הן שבוע 11.

יב · "מה זה 'מטפל לא מצייר בעצמו'?"

דחיפה. אחרי שהסרת סיבוב, כמה מקומות בקוד יודעים שהוא נעלם?

אסטרטגיה. אם המטפל עושה `row.remove()` וגם מסנן את המערך, יש שני מקורות אמת. אם הוא רק מסנן וקורא ל- `render`, יש אחד — ואי אפשר שהמסך והנתונים ייפרדו.

כמעט-פתרון. גוף המאזין הוא שלוש שורות: למצוא את הכפתור, לסנן את `rounds`, לקרוא ל- `render()`. **חריג אחד מותר:** מכה בתוף לא משנה שום נתון, ולכן היא כן נוגעת ב-DOM ישירות ולא קוראת ל- `render` בכלל.